

Project Structure

financial-tracker/

- ├──  package.json # Project dependencies and scripts
- ├──  .env.example # Example environment file (copy to .env)
- ├──  .gitignore # Files to ignore in version control
- ├──  README.md # Complete documentation
- ├──  SETUP_GUIDE.md # Quick 5-minute setup instructions
- ├──  PROJECT_STRUCTURE.md # This file - explains file organization
- ├──  bot.js # Main Telegram bot (Part 1)
 - ├── Handles /personal command
 - ├── Handles /business command
 - ├── Interactive conversations
 - ├── Data validation
 - └── Saves to database
- ├──  database.js # Database operations
 - ├── Creates SQLite database
 - ├── personal_transactions table
 - ├── business_transactions table
 - ├── All query functions
 - └── BAS calculations
- ├──  personal-dashboard.js # Personal finance dashboard (Part 2)
 - ├── Web server on port 3000
 - ├── Monthly summaries
 - ├── Category charts
 - ├── Transaction history
 - └── CSV export
- └──  business-dashboard.js # Business/ATO dashboard (Part 3)
 - ├── Web server on port 3001
 - ├── BAS reporting (G1, G10, G11, G18, 1A)
 - ├── Profit & Loss statements
 - ├── Compliance alerts
 - └── Transaction management

Generated files (created when you run the system):

- ├──  .env # YOUR bot token (keep secret!)
- └──  financial_tracker.db # YOUR financial data (backup regularly!)

File Purposes

Configuration Files

package.json

- Lists all the software packages needed
- Defines commands like `npm start`
- You don't need to edit this

.env.example

- Shows you what your `.env` file should look like
- Copy this and rename to `.env`
- Then add your real bot token

.gitignore

- Tells Git what files to ignore
- Protects your token and database from being shared accidentally
- Important for security!

Documentation Files

README.md

- Complete guide to everything
- Installation instructions
- How to use each feature
- Troubleshooting
- Australian tax information

SETUP_GUIDE.md

- Quick start guide
- Gets you running in 5 minutes
- Simple step-by-step instructions

PROJECT_STRUCTURE.md (this file)

- Explains what each file does
 - Helps you understand the codebase
-

Core Application Files

bot.js - The Telegram Bot

- **Size:** ~500 lines
- **Purpose:** Interactive bot for data entry
- **Features:**
 - `/start` - Welcome message
 - `/personal` - Add personal transactions
 - `/business` - Add business transactions
 - `/summary` - Monthly overview
 - `/help` - Command list
 - `/cancel` - Cancel current transaction

How it works:

1. User sends `/personal` or `/business`
2. Bot asks questions one by one
3. User provides answers via buttons or text
4. Bot validates each input
5. Saves complete transaction to database
6. Confirms with summary

Key Features:

- Inline keyboards (clickable buttons)
 - State management (remembers where user is in conversation)
 - Input validation (checks dates, amounts, etc.)
 - Automatic GST calculation for business
 - ABN tracking for compliance
-

database.js - Database Management

- **Size:** ~400 lines
- **Purpose:** All database operations
- **Tables:**
 - `personal_transactions` - Personal income/expenses
 - `business_transactions` - Business transactions

Functions:

- `initializeDatabase()` - Creates tables
- `addPersonalTransaction()` - Saves personal data
- `addBusinessTransaction()` - Saves business data
- `getPersonalTransactionsByMonth()` - Gets monthly data
- `getBASDataForQuarter()` - Calculates BAS fields
- `getProfitLoss()` - Generates P&L statement
- `getComplianceAlerts()` - Finds compliance issues

Australian Tax Features:

- Automatic GST calculation (10%)
- BAS field calculations (G1, G10, G11, G18, 1A)
- Quarterly reporting (Jul-Sep, Oct-Dec, Jan-Mar, Apr-Jun)
- Financial year handling (July 1 - June 30)
- ABN tracking
- Large transaction flagging (>\$10,000)

personal-dashboard.js - Personal Finance Dashboard

- **Size:** ~800 lines (includes HTML/CSS/JS)
- **Purpose:** View and analyze personal finances
- **Runs on:** <http://localhost:3000>

Features:

1. Summary Cards:

- Total Income
- Total Expenses
- Balance (Income - Expenses)

2. Charts:

- Category Breakdown (pie chart)
- 6-Month Trend (line chart)

3. Transaction Table:

- Searchable
- Filterable by month/year
- Shows all details

4. Export:

- CSV download for Excel/Sheets

How it works:

- Express.js web server
- Single-page application
- API endpoints for data
- Chart.js for visualizations
- Real-time updates

business-dashboard.js - Business/ATO Dashboard

- **Size:** ~1000 lines (includes HTML/CSS/JS)
- **Purpose:** ATO-compliant business reporting
- **Runs on:** <http://localhost:3001>

4 Main Tabs:

1. BAS Reporting

- Select financial year and quarter
- Automatic calculation of all BAS fields:
 - **G1**: Total sales (including GST)
 - **G10**: Capital purchases
 - **G11**: Non-capital purchases
 - **G18**: GST on purchases
 - **1A**: GST on sales
 - **Net GST**: Amount to pay/refund
- Visual cards showing each field
- Expense breakdown chart
- Period information

2. Profit & Loss

- Select date range
- Generates P&L statement:
 - Revenue
 - Cost of Goods Sold (COGS)
 - Gross Profit
 - Operating Expenses
 - Net Profit/Loss
- Visual indication of profit vs loss
- Detailed breakdowns

3. Compliance

- Checks for ATO compliance issues:
 - Transactions over \$10,000 (AUSTRAC reporting)
 - Missing ABN for payments >\$75
 - Lists all flagged transactions
 - Calculates potential withholding (47%)
- Traffic light system (green = good, red = needs attention)

4. Transactions

- View all business transactions
 - Filter by month/year
 - Shows:
 - Date, Type, Category
 - Amount, GST, Total
 - ABN status
 - Description
 - Easy to scan and review
-

Data Flow

User → Telegram Bot → database.js → SQLite Database



Dashboard reads ← ← ← ←

Browser displays charts/reports

Example: Adding a Business Transaction

1. **User:** Opens Telegram, sends `/business`
2. **bot.js:** Receives command, starts conversation
3. **bot.js:** Asks "Income or Expense?"
4. **User:** Clicks "Expense"
5. **bot.js:** Shows expense categories
6. **User:** Clicks "Materials"
7. **bot.js:** Asks for date
8. **User:** Types "today"
9. **bot.js:** Asks for amount
10. **User:** Types "1000"
11. **bot.js:** Calculates GST (\$100), asks about ABN
12. **User:** Clicks "Yes"
13. **bot.js:** Asks for description
14. **User:** Types "Paint supplies from Bunnings"
15. **bot.js:** Validates all data
16. **database.js:** Saves to `business_transactions` table
17. **bot.js:** Confirms with summary

Viewing in Dashboard

1. **User:** Opens browser to <http://localhost:3001>
 2. **browser:** Requests data from dashboard
 3. **business-dashboard.js:** Receives request
 4. **database.js:** Queries database
 5. **business-dashboard.js:** Sends data as JSON
 6. **browser:** Receives data
 7. **JavaScript:** Renders charts and tables
 8. **User:** Sees beautiful visualization!
-

Code Organization

bot.js Structure

javascript

// 1. Setup and configuration

const TelegramBot = **require**(...)

const db = **require**(...)

// 2. Constants

const PERSONAL_CATEGORIES = [...]

const BUSINESS_INCOME_CATEGORIES = [...]

// 3. Helper functions

function createInlineKeyboard() {...}

function formatCurrency() {...}

// 4. Command handlers

bot.onText(/start/, ...)

bot.onText(/personal/, ...)

bot.onText(/business/, ...)

// 5. Callback handlers (button clicks)

bot.on('callback_query', ...)

// 6. Message handlers (text input)

bot.on('message', ...)

// 7. Error handling

bot.on('polling_error', ...)

database.js Structure

javascript

// 1. Database connection

```
const Database = require('better-sqlite3')
```

```
const db = new Database(...)
```

// 2. Initialize function

```
function initializeDatabase() {...}
```

// 3. Personal transaction functions

```
function addPersonalTransaction() {...}
```

```
function getPersonalTransactionsByMonth() {...}
```

// 4. Business transaction functions

```
function addBusinessTransaction() {...}
```

```
function getBASDataForQuarter() {...}
```

```
function getProfitLoss() {...}
```

// 5. Compliance functions

```
function getComplianceAlerts() {...}
```

// 6. Export functions

```
module.exports = {...}
```

Dashboard Structure

javascript

```
// 1. Express setup
const express = require('express')
const app = express()

// 2. API endpoints
app.get('/api/summary', ...)
app.get('/api/transactions', ...)
app.get('/api/export', ...)

// 3. HTML page
app.get('/', (req, res) => {
  const html = `
    <!DOCTYPE html>

    ...
    <script>
      // Frontend JavaScript
      async function loadData() {...}
      function updateCharts() {...}
    </script>
  `
  res.send(html)
})

// 4. Start server
app.listen(PORT, ...)
```

Technology Stack

Backend (Node.js)

- **node-telegram-bot-api** - Telegram bot framework
- **better-sqlite3** - Fast SQLite database
- **express** - Web server framework
- **dotenv** - Environment variable management

Frontend (Browser)

- **HTML5** - Structure
- **CSS3** - Styling (no framework needed!)
- **JavaScript** - Interactivity
- **Chart.js** - Beautiful charts

Database

- **SQLite** - Simple, fast, reliable
 - **No server required** - Just a file
 - **ACID compliant** - Data integrity
 - **Perfect for this use case**
-

Where Things Are Stored

Your Bot Token

- **File:** `.env`
- **Location:** Project root
- **Format:** `TELEGRAM_BOT_TOKEN=your_token_here`
- **Security:** NEVER commit to Git, NEVER share

Your Financial Data

- **File:** `financial_tracker.db`
- **Location:** Project root
- **Format:** SQLite database (binary)
- **Security:** Backup regularly!

Transaction Data

- **Table:** `personal_transactions`
 - Stores all personal income/expenses
 - 7 columns: id, date, amount, category, description, payment_method, created_at
- **Table:** `business_transactions`
 - Stores all business transactions
 - 9 columns: id, date, amount, type, category, description, gst_amount, has_abn, receipt_url, created_at

Australian Tax Compliance

BAS Fields Explained

G1 - Total Sales (including GST)

- All income transactions
- Amount + GST
- Example: \$1000 sale + \$100 GST = \$1100 (reported in G1)

1A - GST on Sales

- GST collected from customers
- 10% of sales
- Example: Sold \$1000 worth → collected \$100 GST (reported in 1A)

G10 - Capital Purchases

- Equipment, tools (long-term assets)
- In this system: "Equipment Hire" category
- Separated for depreciation tracking

G11 - Non-Capital Purchases

- Operating expenses
- Materials, subcontractors, fuel, etc.
- Everything except capital purchases

G18 - GST on Purchases

- GST you paid to suppliers
- Can be claimed back
- Example: Bought \$1000 materials + \$100 GST (GST goes in G18)

Net GST

- Formula: $1A - G18$
- Positive = You owe ATO
- Negative = ATO owes you refund

Example BAS Quarter

Q1 (Jul-Sep 2024):

- Sold 3 projects: \$30,000 + \$3,000 GST
- Materials: \$10,000 + \$1,000 GST
- Subcontractors: \$8,000 + \$800 GST

BAS Fields:

- G1: \$33,000 (total sales with GST)
 - 1A: \$3,000 (GST collected)
 - G11: \$18,000 (materials + subs)
 - G18: \$1,800 (GST paid)
 - Net GST: \$3,000 - \$1,800 = \$1,200 (pay to ATO)
-

Modifying the Code

Adding a New Personal Category

In **bot.js**, find:

```
javascript

const PERSONAL_CATEGORIES = [
  'Income', 'Groceries', 'Utilities', 'Entertainment', 'Transport', 'Other'
];
```

Add your category:

```
javascript

const PERSONAL_CATEGORIES = [
  'Income', 'Groceries', 'Utilities', 'Entertainment', 'Transport',
  'Healthcare', // NEW CATEGORY
  'Other'
];
```

That's it! The bot will automatically show the new button.

Adding a New Business Category

In **bot.js**, find:

```
javascript
```

```
const BUSINESS_EXPENSE_CATEGORIES = [  
  'Materials', 'Subcontractors', ...  
];
```

Add your category:

```
javascript
```

```
const BUSINESS_EXPENSE_CATEGORIES = [  
  'Materials', 'Subcontractors', 'Equipment Hire', 'Vehicle', 'Fuel',  
  'Insurance', 'Marketing', 'Office Supplies', 'Professional Fees',  
  'Training', // NEW CATEGORY  
  'Other'  
];
```

Changing GST Rate (if it ever changes)

In `bot.js`, find:

```
javascript
```

```
state.data.gst_amount = amount * 0.10; // 10% GST
```

Change to new rate:

```
javascript
```

```
state.data.gst_amount = amount * 0.15; // 15% GST (example)
```

Performance & Scalability

Current Limits

- **Transactions:** Unlimited (SQLite can handle millions)
- **Users:** 1 (designed for personal use)
- **Concurrent requests:** ~100 (more than enough for dashboards)
- **Data size:** Limited by disk space

If You Need More

- **Multiple users:** Add user_id field to tables
 - **Cloud deployment:** Use PostgreSQL instead of SQLite
 - **Mobile app:** Dashboards are responsive and work on phones
 - **API access:** All endpoints are RESTful and documented
-

Future Enhancements (Not Yet Implemented)

Ideas for version 2.0:

- 📷 Receipt photo upload via Telegram
 - ✏️ Edit/delete transactions from bot
 - 👥 Multi-user support
 - 📱 Native mobile app
 - ☁️ Automatic backups to cloud
 - 📊 Project tracking (link expenses to projects)
 - 📄 Invoice generation
 - 📧 Email reports
 - 🔔 Reminder notifications
 - 📈 Advanced analytics
-

Getting Help

1. **Check README.md** - Comprehensive documentation
 2. **Check SETUP_GUIDE.md** - Quick troubleshooting
 3. **Check error messages** - Terminal shows helpful errors
 4. **Check browser console** - F12 in browser shows JavaScript errors
 5. **Restart everything** - Solves 80% of issues!
-

You now understand the entire system! 🎉

Feel free to explore the code - everything is heavily commented to help you learn!